Curso Tendencias Aplicadas de Ingenería de Software

APIs REST

¿Qué es una API?

- Una API (Application Programming Interface) es un contrato que define cómo interactúan componentes o sistemas de software.

- Funciones principales
  - Exposición de funcionalidades.
  - Intercambio de datos.
  - Integración entre sistemas.

- Ejemplos
  - Sistemas de pagos.
  - Plataformas de e-commerce.
  - Arquitecturas de microservicios.

- Una API no es solamente una interfaz técnica; representa capacidades de negocio que pueden ser reutilizadas por distintos consumidores.

Introducción a REST

- REST (Representational State Transfer), es un estilo arquitectónico propuesto por Roy Fielding (2000) para sistemas distribuidos.

- REST no es un protocolo ni una tecnología.
- REST es una forma de diseñar APIs.
- Es un conjunto de reglas simples para organizar la comunicación.
- REST usa HTTP de manera ordenada y consistente.
- Es un conjunto de restricciones arquitectónicas que permiten construir sistemas:
  - Escalables.
  - Evolucionables.
  - Independientes.
  - Interoperables.

- REST aprovecha la infraestructura existente de Internet:
  - HTTP
  - URI
  - Códigos de estado
  - Caché

Introducción a REST

- Principios fundamentales
  - Cliente-Servidor.
  - Stateless.
  - Cacheable.
  - Uniform Interface.
  - Layered System.
- Beneficios
  - Simplicidad.
  - Escalabilidad.
  - Interoperabilidad.
  - Bajo acoplamiento.

¿Qué hace que una API sea REST?

- Una API es REST si sigue estas ideas:
  - Usa HTTP correctamente
  - Trabaja con recursos (no acciones)
  - Es stateless (sin memoria de sesión)
  - Usa URLs claras y consistentes

Restricciones REST y Propiedades Arquitectónicas

- Restricción 1: Cliente-Servidor
- Separación entre:
  - Presentación
  - Procesamiento de negocio
  - Persistencia

- Beneficios
  - Evolución independiente.
  - Mejor mantenibilidad.
  - Reutilización.

Restricciones REST y Propiedades Arquitectónicas

- Restricción 2: Stateless
- Cada solicitud debe contener toda la información necesaria.
- El servidor no mantiene contexto de sesión.
- Beneficios
  - Escalabilidad horizontal.
  - Tolerancia a fallos.
  - Balanceo de carga sencillo.
- Desventaja
  - Mayor volumen de datos transmitidos.

Recursos y Representaciones

- REST modela entidades del dominio como recursos.
- Ejemplos
  - /api/customers/123
  - /api/orders/456
  - /api/products/789
- Representaciones
  - JSON
  - XML
  - YAML
- Ejemplo JSON

```
{
        "id": 123,
        "name": "Juan P",
        "email": "juan@domain.com"
}
```

Tácticas de Diseño REST

- Diseño orientado a recursos

  ✔ Correcto:
  /customers/123/orders

  ✖ Incorrecto:
  /getCustomerOrders

- Uso consistente de HTTP
  - GET: para consultar
  - POST: para crear
  - PUT: para actualizar
  - PATCH: para actualizar parcialmente
  - DELETE: para Eliminar

- Beneficio
  - Mayor coherencia y facilidad de adopción.

Tácticas de Diseño REST

- Consistencia de Recursos
- Todos los recursos deben seguir convenciones homogéneas.
- Ejemplo:
  - /customers
  - /products
  - /orders
- No:
  - /customerList
  - /productsData
  - /orderManagement

Tácticas de Diseño: Manejo de Errores

- Mala práctica: HTTP 200 para todos los casos.
- Buena práctica: HTTP correcto + mensaje estructurado.
- Ejemplos código HTTP:
  - 200 OK
  - 201 Created
  - 400 Bad Request
  - 404 Not Found
  - 409 Conflict
  - 500 Internal Server Error
- Respuesta estandarizada

```
{
        "error": "CustomerNotFound",
        "message": "Customer 123 does not exist"
}
```

- Beneficio
  - Mejor experiencia para desarrolladores.

Rol Estratégico de las APIs

- Las APIs permiten:
- Expansión de Ecosistemas
- Ejemplos:
  - Open Banking.
  - Marketplaces.
  - Plataformas de pagos.
- Desarrolladores externos construyen nuevas aplicaciones usando capacidades existentes.
- Monetización
  - La API puede convertirse en una fuente directa de ingresos.

Documentación como Activo Estratégico

- Una API técnicamente excelente puede fracasar si es difícil de comprender.
- La experiencia del desarrollador (Developer Experience - DX) se vuelve crítica.
- Componentes esenciales
  - Introducción funcional.
  - Casos de uso.
  - Autenticación.
  - Modelos de datos.
  - Ejemplos ejecutables.
  - Errores comunes.

¿Qué es la especificación de una API?

- La especificación es un documento que describe:
  - Qué hace la API
  - Cómo se usa
  - Qué datos recibe
  - Qué datos devuelve

- Es un contrato entre sistemas.
- Una API REST bien especificada incluye:
  - 1. Endpoints: URLs disponibles
    Ej: /users, /orders
  - 2. Métodos HTTP
    GET, POST, PUT, DELETE

¿Qué es la especificación de una API?

- 3. Parámetros
  - Query params
  - Path params
- 4. Request body
  - Datos enviados
- 5. Responses
  - Formato de respuesta
  - Códigos HTTP

Ejemplo simple de especificación

- Endpoint: GET /users/{id}
- Descripción:
  - Obtiene información de un usuario
- Parámetros:
  - id (path): identificador del usuario
- Respuesta:

```
{
    "id": 1,
    "name": "Ana",
    "email": "ana@email.com"
}
```

- Códigos:
  - 200 OK
  - 404 Not Found

OpenAPI

- OpenAPI es un estándar para describir APIs REST.
- Permite escribir la API en formato estructurado (YAML o JSON).
- A partir de un documento OpenAPI es posible:
  - Generar documentación.
  - Generar mocks.
  - Ejecutar pruebas automáticas.
  - Validar contratos.
- Impacto Organizacional
  - Reduce significativamente la deuda técnica asociada a integraciones.

Ejemplo simple de OpenAPI

```
info:
  title: User API
  version: 1.0.0
  description: API simple para gestión de usuarios

servers:
  - url: https://api.ejemplo.com/v1

paths:
  /users/{id}:
    get:
      summary: Obtener usuario por ID
      description: Retorna la información de un usuario específico
      operationId: getUserById

      parameters:
        - name: id
          in: path
          required: true
          description: Identificador único del usuario
          schema:
            type: integer
            example: 1
```

Ejemplo simple de OpenAPI

```
responses:
  "200":
    description: Usuario encontrado correctamente
    content:
      application/json:
        schema:
          type: object
          properties:
            id:
              type: integer
              example: 1
            name:
              type: string
              example: Ana
            email:
              type: string
              example: ana@email.com

  "404":
    description: Usuario no encontrado
    content:
      application/json:
        schema:
          type: object
          properties:
            error:
              type: string
              example: USER_NOT_FOUND
            message:
              type: string
              example: No se encontró el usuario con ese ID
```

OpenAPI

- info: información general de la API
- servers: dónde vive la API
- paths: endpoints disponibles
- /users/{id}: recurso con parámetro dinámico
- get: operación HTTP
- parameters: entradas de la API
- responses: posibles salidas

¿Por qué es importante la especificación?

- Sin especificación:
  - Cada equipo interpreta diferente la API
  - Errores de integración
  - Documentación desactualizada

- Con especificación:
  - Todos entienden lo mismo
  - Se puede automatizar desarrollo
  - Se reduce ambigüedad

¿Qué significa invocar una API?

- Invocar una API significa:
  - Enviar una petición HTTP a un servidor
  - Recibir una respuesta estructurada (normalmente JSON)

  Ejemplo de flujo:
  - Cliente -> API Servidor -> Respuesta
- Ejemplos de clientes:
  - curl (terminal)
  - Postman (GUI)
  - Navegador web (GET requests)
  - Aplicaciones móviles
  - Backend servers (Java, Python, Node.js)
- Una API no depende de una sola herramienta, puede ser consumida por muchos tipos de clientes.

¿Por qué existen varios clientes de API?

- Porque cada contexto necesita algo distinto:

Cliente	**Uso principal**
curl	pruebas rápidas en terminal
Postman	Testing Visual
Browser	Testing o acceso a APIs
Mobile app	Uso de API para funcionalidad requerida en app
Backend	Idem a la anterior (integración entre sistemas)

- La API es independiente del cliente

¿Qué es curl?

- curl (Client URL) es una herramienta de línea de comandos que permite:
  - Enviar requests HTTP
  - Probar APIs sin interfaz gráfica
  - Simular clientes reales
- En simple:
  - Es una forma de “hablar con una API desde la terminal”.
- Muy usada en:
  - Backend
  - DevOps
  - Testing
  - Debugging

Ejemplo básico GET con curl

- curl -X GET https://api.ejemplo.com/users/10
- Respuesta:

```
{
  "id": 10,
  "name": "Ana P",
  "email": "ana@email.com"
}
```

- GET con headers
  - curl -X GET https://api.ejemplo.com/v1/users/10 \
    -H "Accept: application/json"

Ejemplo básico POST con curl

- POST para crear recursos, en este caso un usuario

- curl -X POST https://api.ejemplo.com/users \
  -H "Content-Type: application/json" \
  -d '{
  "name": "Carlos L",
  "email": "carlos@email.com"
  }'

Ejemplo básico PUT con curl

- POST para actualización total de recursos, en este caso el usuario de identificador 10

```
curl -X PUT https://api.ejemplo.com/users/10 \
-H "Content-Type: application/json" \
-d '{
     "name": "Ana P,
      "email": "ana.nuevo@email.com"
}'
```

Ejemplo básico PATCH con curl

- PATCH para actualizar parcial de recursos, en este caso el usuario de identificador 10

```
curl -X PATCH https://api.ejemplo.com/users/10 \
-H "Content-Type: application/json" \
-d '{
      "email": "nuevo@email.com"
}'
```

Ejemplo básico DELETE con curl

- PATCH para eliminar recursos, en este caso el usuario de identificador 10

  curl -X DELETE https://api.ejemplo.com/users/10

REST en Arquitecturas de Microservicios

- Cada microservicio expone capacidades mediante APIs.
- Beneficios:
  - Bajo acoplamiento.
  - Independencia de despliegue.
  - Escalabilidad independiente.
- Desafíos:
  - Latencia.
  - Versionado.
  - Consistencia distribuida.

REST en Arquitecturas de Microservicios

- Las REST APIs son el mecanismo más común de comunicación síncrona entre microservicios.
- Cada microservicio expone una API que define:
  - Qué recursos ofrece (usuarios, órdenes, productos)
  - Qué operaciones se pueden realizar (GET, POST, PUT, DELETE)
  - Qué formato de datos se utiliza (JSON en la mayoría de los casos)
  - Qué reglas de acceso existen
- La API no es un detalle de implementación, es el contrato formal del servicio.
- En este contexto, la API define la frontera entre sistemas independientes.

REST en Arquitecturas de Microservicios

- REST no es un protocolo ni una librería, sino un estilo arquitectónico basado en restricciones.

- En microservicios, REST se usa porque:
  - se apoya en HTTP (infraestructura existente)
  - es simple de implementar
  - es ampliamente interoperable
  - facilita integración entre equipos y organizaciones

- Sin embargo, su uso implica aceptar restricciones importantes:
  - comunicación stateless
  - latencia de red
  - consistencia eventual en sistemas distribuidos

Problemas de acoplamiento en APIs REST

- Uno de los mayores riesgos en microservicios es el acoplamiento entre servicios a través de APIs.
- Esto ocurre cuando:
  - muchos servicios dependen de un mismo endpoint
  - los cambios en una API obligan a modificar múltiples servicios
  - existe dependencia fuerte en la estructura de datos de otro servicio
- Se forma un “distributed monolith”, donde los servicios parecen independientes pero no lo son realmente.
- Esto contradice el objetivo principal de microservicios: independencia.

Latencia y costo de llamadas REST

- Cada llamada REST en un sistema distribuido implica:
  - serialización de datos (JSON)
  - transmisión por red
  - procesamiento en el servidor
  - deserialización de respuesta
- Cuando existen cadenas de llamadas la latencia se acumula progresivamente.
- En microservicios, el rendimiento no depende solo del código, sino del número de llamadas entre servicios.

REST APIs y propiedad de datos

- En microservicios se aplica el principio de data ownership:
  - Cada servicio es dueño exclusivo de sus datos.

- Esto significa: no se accede directamente a bases de datos de otros servicios
- cualquier acceso a datos externos debe hacerse mediante APIs

- Esto es clave para mantener independencia y evitar acoplamiento a nivel de datos.

- Sin este principio, los microservicios dejan de ser independientes.

API-first en microservicios

- El enfoque API-first consiste en diseñar primero el contrato antes de la implementación.
- Proceso:
  - definir OpenAPI specification
  - validar con equipos consumidores
  - generar mocks
  - desarrollar servicios en paralelo
- Permite desarrollo distribuido entre equipos independientes.

REST APIs como arquitectura del sistema

- En microservicios:
  - la arquitectura emerge de las interacciones entre APIs
- Esto significa:
  - el sistema es una red de contratos
  - el diseño de APIs define el comportamiento global
  - los errores de API se convierten en problemas arquitectónicos
- En microservicios, diseñar APIs es diseñar el sistema completo.

¿Qué es el modelo de información de una API?

- El modelo de información define cómo los conceptos del negocio son representados dentro de una API.
- Antes de diseñar endpoints o escribir código, es necesario responder:
  - ¿Qué entidades existen?
  - ¿Qué atributos tienen?
  - ¿Cómo se relacionan?
  - ¿Qué información puede consultar o modificar un cliente?
- Por ejemplo, en un sistema de e-commerce, las entidades principales son:
  - Cliente
  - Producto
  - Pedido
  - Pago
- La API debe representar estos conceptos de forma consistente.

Dominio de negocio y recursos REST

- REST modela el dominio mediante recursos.
- Dominio: Cliente realiza un pedido.

- Recursos
  - Customer
  - Order
  - Product
  - Payment

- Cada recurso posee:
  - identidad única
  - atributos
  - relaciones con otros recursos

- Ejemplo:

```
{
  "id": 1001,
  "status": "confirmed",
  "total": 150.00
}
```

- Un recurso es una abstracción de una entidad del negocio.

Recursos vs Representaciones

- Una de las ideas más importantes de REST: es el Recurso
- Concepto abstracto.
- Ejemplo:
  - Order #1001
- Representación: forma concreta en que se envía al cliente.
- Ejemplo JSON:

```
{
  "id": 1001,
  "status": "confirmed"
}
```

- El recurso existe independientemente de cómo se represente.

Identificación de recursos

- Todo recurso debe tener una identidad única.
- Normalmente se utiliza una URI:
  - /orders/1001
  - /products/50
  - /customers/123
- Características deseables:
  - única
  - permanente
  - independiente de reglas de negocio
- Las URIs identifican recursos, no operaciones.

Atributos de un recurso

- Cada recurso contiene información relevante.
- Ejemplo: Producto

```
{
  "id": 10,
  "name": "Notebook",
  "price": 1200,
  "stock": 15
}
```

Identificación de recursos

- Los recursos rara vez existen aislados.
- Ejemplo: Un Customer realiza muchas ordenes (Orders) que tienen muchos productos (Productos)
- Relaciones típicas:
  - One to One
  - One To Many
  - Many to Many
- Ejemplos REST:
  - /customers/123/orders
  - /orders/1001/products
- El modelo de información incluye tanto entidades como relaciones.

Navegación entre recursos

- REST permite navegar entre recursos relacionados.
- Ejemplo: GET /customers/123
- Respuesta:

```
{
  "id": 123,
  "name": "Ana"
}
```

- Luego: GET /customers/123/orders
- La API refleja la estructura lógica del dominio.

Representación en JSON

- JSON es el formato más utilizado en APIs REST.
- Ejemplo:

```
{
  "id": 1001,
  "status": "confirmed",
  "total": 150.00
}
```

- Ventajas:
  - legible
  - compacto
  - ampliamente soportado
- Por esta razón se convirtió en el estándar de facto para APIs modernas.

Colecciones de recursos

- Las APIs suelen exponer colecciones.
- Ejemplo: GET /products
- Respuesta:

```
[
  {
    "id": 1,
    "name": "Mouse"
  },
  {
    "id": 2,
    "name": "Keyboard"
  }
]
```

- Una colección representa múltiples instancias de un mismo recurso.

Modelo de información y OpenAPI

- OpenAPI permite formalizar el modelo de información.
- Ejemplo:

```
components:
  schemas:
    Product:
      type: object
      properties:
        id:
          type: integer
        name:
          type: string
        price:
          type: number
```

- El esquema se convierte en una descripción formal del recurso.